



République Algérienne Démocratique et
Populaire
Ministère de l'Enseignement Supérieur et de la
Recherche Scientifique
Université des Sciences et Technologies HOUARI
BOUMEDIENE



NumAnalyse-Application Web Interactive

d'Analyse Numérique

Équipe:

NADRI Yousra Malika

HALI Houda

BOUMECHHOUR Sabrina

OULED LAID Zineb

Encadré par :
BENADJAL Nesrine

Table des matières

1. Introduction.....	3
2. Objectifs du Projet.....	3
3. Architecture du Projet.....	4
3.1 Structure des Fichiers	4
3.2 Technologies Utilisées	4
4. Axe 1 — Résolution d'Équations Non Linéaires.....	5
4.1 Méthode de Dichotomie (Bisection)	5
4.2 Méthode de Newton-Raphson.....	5
4.3 Méthode du Point Fixe	5
5. Axe 2 — Résolution de Systèmes Linéaires.....	5
5.1 Méthodes Directes	6
<i>Élimination de Gauss</i>	6
<i>Décomposition LU</i>	6
<i>Méthode de Cholesky</i>	6
5.2 Méthodes Itératives	6
<i>Méthode de Jacobi</i>	6
<i>Méthode de Gauss-Seidel</i>	6
6. Axe 3 — Interpolation, Approximation et Optimisation.....	6
6.1 Interpolation Polynomiale	7
6.2 Approximation par Moindres Carrés.....	7
6.3 Descente de Gradient.....	7
7. Fonctionnalités Interactives.....	8
8. Conclusion.....	9

1. Introduction

L'analyse numérique constitue l'une des disciplines fondamentales de l'informatique scientifique et des mathématiques appliquées. Elle fournit les outils algorithmiques et mathématiques permettant de résoudre, de manière approchée mais rigoureusement contrôlée, des problèmes qui ne possèdent pas toujours de solution analytique exacte. Parmi ces problèmes figurent les équations non linéaires, les systèmes linéaires de grande taille, les problèmes d'interpolation et d'approximation de fonctions, ou encore l'optimisation par descente de gradient.

Dans un contexte pédagogique, la compréhension de ces méthodes est souvent entravée par leur caractère abstrait. Les étudiants peinent à saisir intuitivement la dynamique de convergence, les conditions de stabilité, ou encore l'impact du choix de l'initialisation sur la qualité du résultat. C'est précisément dans cette optique qu'a été conçu le présent projet.

Ce projet a pour objectif principal le développement d'une application web interactive permettant à l'utilisateur d'expérimenter en temps réel l'ensemble des méthodes numériques étudiées dans le module. Grâce à une interface claire et pédagogique, l'application transforme des algorithmes abstraits en outils visuels et interactifs, favorisant ainsi une appropriation profonde des concepts théoriques.

2. Objectifs du Projet

L'application a été conçue en réponse à un ensemble d'objectifs pédagogiques et techniques clairement définis, visant à maximiser l'utilité éducative tout en garantissant la robustesse de l'outil :

- Permettre l'expérimentation en temps réel de toutes les méthodes numériques du module, avec des paramètres librement configurables par l'utilisateur.
- Visualiser graphiquement la convergence de chaque algorithme, afin de rendre palpable la dynamique itérative de chaque méthode.
- Afficher les étapes intermédiaires détaillées : tableaux d'itérations, matrices décomposées, polynômes construits, etc.
- Comparer les méthodes d'un même axe sur des données identiques et recommander automatiquement la plus adaptée au problème posé.
- Proposer une architecture modulaire Python/Flask séparant clairement le backend algorithmique du frontend interactif.

Ces objectifs s'inscrivent dans une démarche d'apprentissage actif, où l'étudiant n'est plus passif face à un cours magistral, mais devient acteur de ses propres expérimentations numériques.

3. Architecture du Projet

3.1 Structure des Fichiers

Le projet adopte une architecture modulaire claire, où chaque axe mathématique correspond à un package Python indépendant, favorisant la maintenabilité et l'extensibilité du code :

Module / Fichier	Rôle
utils.py	Fonctions partagées : parse_expr(), safe_eval()
Axe1/methodes.py	Bisection, Newton-Raphson, Point fixe
Axe2/methodes_directes/	Gauss, LU, Cholesky
Axe2/methodes_indirectes/	Jacobi, Gauss-Seidel
Axe3/methodes.py	Interpolation, moindres carrés, gradient
app.py	Routes Flask uniquement (aucun calcul embarqué)
numanalyse.html	Interface utilisateur unique (SPA)

3.2 Technologies Utilisées

Backend	Python 3 avec Flask comme micro-framework web. Les calculs numériques s'appuient sur NumPy pour l'algèbre linéaire et les tableaux, et sur SymPy pour la dérivation symbolique et l'évaluation d'expressions. Chaque route Flask reçoit un objet JSON, délègue le calcul au module correspondant et retourne la réponse en JSON.
Frontend	Page HTML/CSS/JavaScript sans framework externe. L'interface est organisée en trois onglets correspondant aux trois axes. Les visualisations sont réalisées avec Chart.js pour les courbes et tableaux de données interactifs.

4. Axe 1 — Résolution d'Équations Non Linéaires

Le premier axe est consacré à la résolution numérique d'équations de la forme $f(x) = 0$, où f est une fonction réelle continue. L'utilisateur saisit librement une expression mathématique ainsi qu'un intervalle de recherche. Trois méthodes distinctes, aux propriétés de convergence complémentaires, ont été implémentées.

4.1 Méthode de Dichotomie (Bisection)

La méthode de dichotomie repose sur le théorème des valeurs intermédiaires : si f est continue sur $[a, b]$ et que $f(a) \cdot f(b) < 0$, il existe au moins une racine dans cet intervalle. L'algorithme divise répétitivement l'intervalle en deux sous-intervalles, en conservant celui qui contient un changement de signe.

La convergence est linéaire et garantie, avec une erreur bornée par $(b-a)/2^n$ après n itérations. Cette robustesse en fait une méthode de référence, quoique plus lente que les alternatives.

4.2 Méthode de Newton-Raphson

Newton-Raphson utilise la tangente à la courbe pour affiner l'approximation selon la formule de récurrence : $x_{n+1} = x_n - f(x_n)/f'(x_n)$. La convergence est quadratique — extrêmement rapide — sous réserve que x_0 soit suffisamment proche de la racine et que $f'(x) \neq 0$.

L'application calcule automatiquement f' et f'' via SymPy, et estime l'erreur a priori par la formule de Kantorovitch, offrant ainsi une garantie théorique de convergence à l'utilisateur.

4.3 Méthode du Point Fixe

Le problème $f(x) = 0$ est reformulé en une équation de point fixe $x = g(x)$. Partant d'une valeur initiale x_0 , la suite définie par $x_{n+1} = g(x_n)$ converge vers la solution si $|g'(x)| < 1$ au voisinage de celle-ci.

L'application affiche à chaque itération la valeur de $|g'(x_n)|$, permettant à l'utilisateur de diagnostiquer en temps réel la condition de convergence et d'ajuster sa fonction g si nécessaire.

5. Axe 2 — Résolution de Systèmes Linéaires

Le second axe traite la résolution numérique de systèmes de la forme $Ax = b$, où A est une matrice carrée d'ordre n et b un vecteur second membre. L'utilisateur définit interactivement la taille de la matrice, puis saisit ses coefficients. Deux grandes familles de méthodes ont été implémentées.

5.1 Méthodes Directes

Élimination de Gauss

L'élimination de Gauss transforme le système en un système triangulaire supérieur par des opérations élémentaires sur les lignes (pivot partiel), suivi d'une phase de remontée. L'application retourne la matrice échelonnée U , permettant de visualiser chaque étape de la triangularisation.

Décomposition LU

La factorisation LU décompose A en un produit $L \cdot U$, où L est triangulaire inférieure (diagonale unité) et U triangulaire supérieure. La résolution s'effectue en deux passes successives : $Ly = b$, puis

$Ux = y$. Les matrices L et U sont affichées explicitement pour vérification.

Méthode de Cholesky

La méthode de Cholesky est réservée aux matrices symétriques définies positives. Elle exploite la factorisation $A = LL^T$, plus efficace que la décomposition LU pour ce cas particulier, réduisant le nombre d'opérations d'un facteur deux. La matrice L triangulaire est affichée pour contrôle.

5.2 Méthodes Itératives

Méthode de Jacobi

À chaque itération, chaque composante x_i est recalculée en utilisant exclusivement les valeurs de l'itération précédente. Cette approche est naturellement parallélisable, mais présente une convergence plus lente que Gauss-Seidel pour la plupart des matrices.

Méthode de Gauss-Seidel

Variante améliorée de Jacobi, Gauss-Seidel utilise immédiatement les nouvelles valeurs dès qu'elles sont calculées. Pour les matrices à dominance diagonale stricte, cette méthode converge généralement en environ deux fois moins d'itérations que Jacobi.

L'application compare automatiquement les deux méthodes itératives sur les mêmes données et désigne la plus rapide et la plus précise selon le résidu $\|Ax - b\|_2$.

6. Axe 3 — Interpolation, Approximation et Optimisation

Le troisième et dernier axe regroupe trois problèmes distincts mais complémentaires : l'interpolation polynomiale exacte, l'approximation par moindres carrés, et l'optimisation par descente de gradient.

6.1 Interpolation Polynomiale

Étant donné $n+1$ points distincts (x_i, y_i) , l'interpolation polynomiale construit l'unique polynôme de degré $\leq n$ passant exactement par tous les points. Deux formulations mathématiquement équivalentes ont été implémentées :

- Interpolation de Lagrange : combinaison des polynômes de base $L_i(x)$ pondérés par les ordonnées y_i .
- Interpolation de Newton : construction incrémentale via les différences divisées, plus efficace lors de l'ajout de nouveaux points.

L'application affiche le tableau des différences divisées, les deux polynômes construits sous forme symbolique, et vérifie leur équivalence en calculant l'écart maximal entre les deux représentations.

6.2 Approximation par Moindres Carrés

Contrairement à l'interpolation, l'approximation par moindres carrés ne cherche pas à passer exactement par tous les points, mais à minimiser la somme des carrés des écarts entre les données et le modèle polynomial. Deux modes sont disponibles :

Mode Discret	Minimisation de $\sum (y_i - P(x_i))^2$. Le polynôme optimal est obtenu par résolution du système normal $A^T A c = A^T y$, où les colonnes de A sont les puissances successives des abscisses.
Mode Continu	Minimisation de $\int (f(x) - P(x))^2 dx$ sur $[a, b]$. Les intégrales impliquées sont calculées symboliquement par SymPy, garantissant une précision exacte.

6.3 Descente de Gradient

La descente de gradient est une méthode d'optimisation itérative qui minimise une fonction f en suivant à chaque pas la direction opposée au gradient local : $x_{n+1} = x_n - \alpha \cdot \nabla f(x_n)$, où α est le pas d'apprentissage.

Deux modes d'utilisation sont proposés : l'optimisation en dimension 1 avec affichage de la trajectoire sur la courbe, et l'optimisation en dimension 2 avec visualisation de la trajectoire dans l'espace.

L'application détecte automatiquement toute divergence (valeur non finie) et avertit l'utilisateur d'ajuster le pas α .

Paramètre α	Effet observé
α trop grand	Divergence — oscillations croissantes autour du minimum
α optimal	Convergence rapide et stable vers le minimum
α trop petit	Convergence très lente — nombreuses itérations nécessaires

7. Fonctionnalités Interactives

Axe 1 – Résolution d'Équations

Sélection de méthode : Dichotomie, Newton, Point Fixe

Saisie de fonction : Champ texte avec Paramètres : Intervalle, tolérance, itération maximale

Résultats : Tableau des itérations + graphique de convergence

Recommandation : Système expert suggérant la meilleure méthode

Axe 2 - Systèmes Linéaires

Choix de méthode : Gauss, Decomposition LU , Decomposition Cholesky, Jacobi, Gauss-Seidel

Saisie de matrice : Interface pour définir la matrice

Résultats : Tableau avec solution,matrice, résidu, itérations...

Comparaison et autre options

Axe 3 - Interpolation et Optimisation

Choix de méthode : Interpolation de Lagrange, Newton **Approximation :** Moindres carrés

Résultats : Polynome interpolateur, Table des différences divisées, graphe...

Comparaison interpolation Lagrange / Newton

Comparaison interpolation / Approximation

8. Conclusion

Ce projet a permis de concevoir et de développer une application web interactive complète dédiée à l'exploration pédagogique des méthodes d'analyse numérique. En couvrant trois axes fondamentaux — la résolution d'équations non linéaires, la résolution de systèmes linéaires, et l'interpolation/approximation/optimisation — l'application offre un cadre unifié et cohérent pour l'expérimentation algorithmique.

L'un des apports majeurs de ce travail réside dans la mise en place d'un module de comparaison automatique, permettant à l'utilisateur de confronter plusieurs méthodes sur les mêmes données et d'en tirer des conclusions objectives quant à leur efficacité respective. Cette fonctionnalité transforme l'application en véritable outil d'aide à la décision numérique.

Sur le plan architectural, le choix d'une séparation stricte entre le backend algorithmique (Python/Flask/NumPy/SymPy) et le frontend interactif (HTML/JavaScript/Chart.js) s'est avéré particulièrement judicieux. Cette modularité facilite l'extension de l'application à de nouveaux algorithmes, et garantit une maintenabilité à long terme du projet.

Du point de vue pédagogique, la visualisation graphique de la convergence et l'affichage des étapes intermédiaires constituent des leviers essentiels pour ancrer la compréhension intuitive des méthodes numériques. L'étudiant peut ainsi observer concrètement comment un algorithme converge — ou diverge — selon les paramètres choisis, et développer une sensibilité algorithmique précieuse pour la pratique de l'ingénierie numérique.